

28-Introduction to Pandas and Advanced NumPy Array Operations

Introduction

This lecture focuses on advanced NumPy array operations and an introduction to the Pandas package, essential for handling tabular data in Python.

Iterating Over Arrays

- Iterating over one-dimensional arrays uses simple loops (e.g., `for i in array`).
- For multi-dimensional arrays, iteration by rows comes first, with an additional nested loop to access elements within each row.
- Flattening an array (via functions like `np.nditer`) allows iteration over all elements in row-wise ('C' order) or column-wise ('F' order), matching C or Fortran conventions.

Splitting Arrays

- The `np.split` function divides a one-dimensional array into equal-sized parts if the split number is a factor of array length; errors arise otherwise.
- Splitting at arbitrary indices is possible by passing a list of indices, which results in subarrays of non-uniform sizes.
- For two-dimensional arrays, `np.split` separates along rows by default but can split along columns by setting `axis=1`.

Concatenating Arrays

- One-dimensional arrays are concatenated with `np.concatenate([array1, array2])`.
- Multi-dimensional arrays can be concatenated along rows (`axis=0`, requires matching columns) or columns (`axis=1`, requires matching rows).
- Shape compatibility is crucial for multi-dimensional concatenation.

Array Functions: Maximum, Minimum, Search, and Sort

- `np.max` / `np.min` find the largest or smallest value in an array.
- `np.argmax` / `np.argmin` return the index of the maximum or minimum value.
- `np.sort` sorts arrays; `np.argsort` gives indices that would sort the array.
- `np.where` returns indices of elements matching a condition, while `np.extract` returns actual values matching a condition.
- Logical conditions (and/or) must use NumPy-specific syntax.

Views and Copies

- Views (created by slicing or using `.view()`) link to the original array: changing the view alters the original array.
- Changing the shape of the original does not affect the view's shape.
- Deep copies (via `.copy()`) are independent: modifications do not affect the source array.

Image Array Operations

- Image files are loaded and converted to NumPy arrays for manipulation.
- Cropping is achieved by array slicing (rows = y-axis, columns = x-axis).
- Arrays/images can be split into sections along rows and columns with `np.split`.
- Concatenation rejoins these sections into a reconstructed image.
- Libraries used: `PIL` and `matplotlib` for image manipulation and visualization.

Introduction to Pandas

- Pandas is developed for handling and analyzing tabular data (CSV, Excel, text files, databases) more conveniently than raw arrays.
- Two main data structures:
 - **Series**: 1D labeled array, similar to a column in a table, supports custom indices.
 - **DataFrame**
: 2D table with columns (can be named) and rows, supports customizable row/column labels.
- Pandas was built on top of NumPy and became popular post-2011.

Pandas Series

- Created from lists, dictionaries, etc., with automatic or user-defined indexing.
- User-defined indices are useful for referencing with meaningful labels (e.g., years).
- Slicing and element access are available, but negative indexing is not supported.
- Series can be created from dictionaries; if extra keys are provided, missing values are marked as null.

Pandas DataFrames

- DataFrame creation methods:
 - Reading from CSV/Excel files (`read_csv`, `read_excel`).
 - From dictionaries, list of tuples (each tuple becomes a row), or dictionaries of Series.
- File reading functions have file-type-specific parameters (covered further in classes).
- DataFrames support column access by both attribute (`df.column`) and key (`df['column']`) methods.
- Can access multiple columns, but individual element access requires referencing a column and then a row index.

Basic DataFrame Operations

- `.shape` returns row and column counts.
- `.info()` summarizes data types and presence of missing values per column.
- `.head(n)` and `.tail(n)` show the first/last n rows respectively.

- Column-wise operations (e.g., max, min) can use either DataFrame or NumPy functions.
- Addition and deletion of rows/columns, and further operations, will be discussed subsequently.

NumPy Broadcasting

- Broadcasting allows arithmetic between arrays of differing shapes given certain conditions:
 - Dimensions must match or one must be 1 (for the unmatched axis).
 - Examples and failure scenarios are demonstrated (e.g., shapes (3,5) and (1,5) work, (3,5) and (2,5) do not).
 - Special case: operation is possible between 1xN and Nx1 arrays.
- Broadcasting is implicit; errors indicate incompatibility for broadcasting.

Q&A and Practical Notes

- Throughout, student questions clarify differences between views and copies, data access methods, the reason for using Pandas, and the mechanics of array operations.
- Stress is placed on remembering the indexing conventions, file handling, and the interplay between NumPy and Pandas.

Conclusion

- The session provides a foundation for working with array data (NumPy) and tabular data (Pandas) in Python.

- Understanding views/copies, broadcasting, and Pandas' data structures is emphasized as essential for data analysis workflows.

Notes powered by [CraftNote](#)